Reviewer Pack — Minimal Local Ring Norm in Affine Geometry and DPLL Search T…

Entry 4c43b773050b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 11:19:06 UTC

Minimal Local Ring Norm in Affine Geometry and DPLL Search Tree Diameter

Entry ID: `4c43b773050b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 11:19:06 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): DPLL Search Trees

Statement:

> For every CNF φ with at most 40 variables, the minimal local ring norm of the affine hull of its Tseitin representation φ_T is linearly correlated with its DPLL search tree diameter d(φ), such that log(n!) * min_{P ∈ φ_T} |P| = Θ(d(φ)).

Rationale (proposer's reasoning):

> Affine geometry provides a rich algebraic structure to study the complexity of boolean functions. The local ring norm measures the size of prime ideals in the affine hull, which can be related to the branching factors in the DPLL search tree, potentially revealing hidden structures in proof complexity.

Taxonomy category: `AffineGeometryxDPLLSearchTrees` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `bbd223441726cdc7`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For every CNF φ with at most 40 variables, if Pearson correlation coefficient is ≥ 0.8 and p-value < 0.05 from a two-sample t-test between log(n!) * min_{P ∈ φ_T} |P| and d(φ), the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for i in range(1, n + 1):
            clause = [random.choice([-1, 1]) * j for j in range(1, n + 1)]
            if all(clause[i] != -clause[j] for j in range(i)):
                cnf.append(clause)
        return cnf

    def tseitin_representation(cnf):
        literals = set()
        for clause in cnf:
            literals.update(abs(lit) for lit in clause)

        tseitin_vars = list(range(1, len(literals) + 1))
        formulas = []

        for i, literal in enumerate(literals, start=1):
            formulas.append(f"{tseitin_vars[i-1]} <-> ( {' & '.join(str(lit) if lit > 0 else f'¬{a

        return formulas

    def dpll_search_tree_diameter(formulas):
        # Simplified DPLL search tree diameter calculation
        n = len(formulas)
        return n * (n - 1) // 2

    def minimal_local_ring_norm(cnf):
        # Placeholder for actual computation of minimal local ring norm
        return random.random()

    n = random.randint(5, 40)
```

```
    cnf = generate_cnf(n)
    tseitin_formulas = tseitin_representation(cnf)
    td = dpll_search_tree_diameter(tseitin_formulas)
    min_norm = minimal_local_ring_norm(cnf)

    metric_value = math.log(math.factorial(n)) * min_norm
    instances_tested = 1
    n_max = n
    conjecture_holds = False
    counterexample = "mapping_undefined"

    return {
        "metric_name": "log(n!) * min_{P ∈ φ_T} |P|",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='<desc>' first_failing_seed={first_failing_seed}'
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e8c10c0.py", line 78, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e8c10c0.py", line 52, in run_trial
    cnf = generate_cnf(n)
```

```
        ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e8c10c0.py", line 25, in generate_cnf
    if all(clause[i] != -clause[j] for j in range(i)):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e8c10c0.py", line 25, in <genexpr>
    if all(clause[i] != -clause[j] for j in range(i)):
           ~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Re-run the test to ensure it completes without crashing and produces the necessary data for correlation analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	30491
2	propose	ollama_remote	glm4:latest	0	0	16766
3	preregistration	ollama_remote	glm4:latest	0	0	9200
4	novelty	ollama_remote	glm4:latest	0	0	18567
5	novelty	ollama_remote	glm4:latest	0	0	8811
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13760
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12368
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24595
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22864
10	judge	ollama_remote	glm4:latest	0	0	8828

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 166250 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/4c43b773050b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4c43b773050b.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4c43b773050b.tar.gz` (if generated)